

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain BPTT and vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI HAR dataset contains six activities: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING and LAYING. Each activity window contains 128 temporal measurements and nine selected raw inertial channels (three-axis body acceleration, three-axis gyroscope and three-axis total acceleration).

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

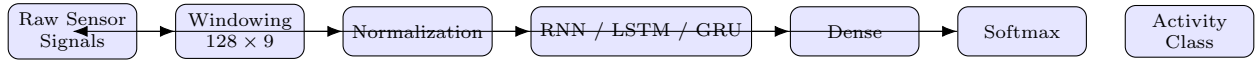
Use the raw inertial signal files rather than only the precomputed 561-feature vectors. A laboratory subset of 1500–3000 windows may be used, while preserving all six classes and approximately balanced representation.

Recommended split:

70% training, 15% validation, 15% testing.

The same preprocessing, split protocol, optimizer, batch size and evaluation procedure must be used for all three recurrent models.

4. Overall Experimental Pipeline



5. Preprocessing

1. Load raw inertial signals.
2. Arrange each observation as 128×9 .
3. Assign one activity label to each window.
4. Encode the six labels.
5. Normalize using training-set statistics only.
6. Create training, validation and test sets.
7. Verify class distribution.

Expected output:

```
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)
Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

6. Temporal Data Visualization

Select representative sequences from at least three activity classes and plot at least three sensor channels against time.

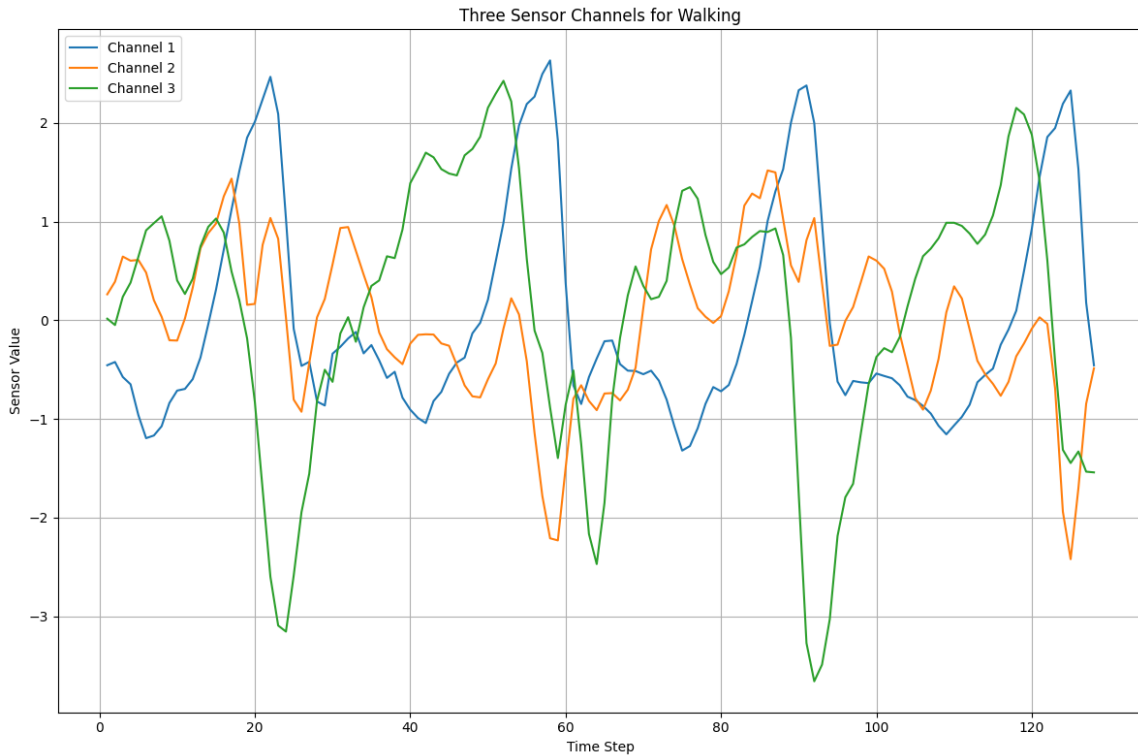


Figure 1: Representative temporal sensor signals for selected human activities.

Inference: he three sensor channels show clear fluctuations over time, with Channel 1 and Channel 3 having the largest variations. Overall, the signals appear to capture repeated walking patterns, with noticeable peaks and dips representing changes in movement or steps.

7. Vanilla RNN Architecture

A Vanilla RNN updates its hidden state as

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

For sequence classification, the final hidden representation can be passed to a classifier.

8. Backpropagation Through Time

BPTT unrolls the RNN across time and propagates the loss gradient backward through the recurrent steps. Repeated multiplication of derivatives can make gradients extremely small or extremely large.

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

describes vanishing gradients, while exploding gradients correspond to very large gradient magnitudes.

For

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2, \quad h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1,$$

the numerical recurrence is

$$h_t = \tanh(0.5x_t + 0.8h_{t-1} + 0.1).$$

The approximate values are

$$h_1 \approx 0.3364, \quad h_2 \approx 0.5474, \quad h_3 \approx 0.5269.$$

9. RNN Implementation

Use:

$$\text{Input}(128 \times 9) \rightarrow \text{SimpleRNN}(32) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(16, \text{ReLU}) \rightarrow \text{Dense}(6) \rightarrow \text{Softmax}.$$

Suggested configuration: Adam, learning rate 10^{-3} , batch size 32, 30 epochs, sparse categorical cross-entropy.

10. LSTM Architecture

LSTM uses forget, input and output gates together with a cell state:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o),$$

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t,$$

$$h_t = o_t \tanh(C_t).$$

Inference: The cell state provides a path for information to persist across many time steps. The gates selectively forget, add and expose information, making LSTM better suited to learning long-term dependencies than a basic RNN.

11. GRU Architecture

GRU uses an update gate and a reset gate:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z),$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate state and final state are

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h),$$

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$

Inference: GRU combines gating into a simpler structure with no separate cell state. It can therefore model temporal dependencies with fewer recurrent parameters than an LSTM of the same hidden size.

12. LSTM and GRU Implementation

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

Only the recurrent layer should change so that the comparison is controlled.

13. Required Training Plots

For each model, generate training/validation loss and accuracy.

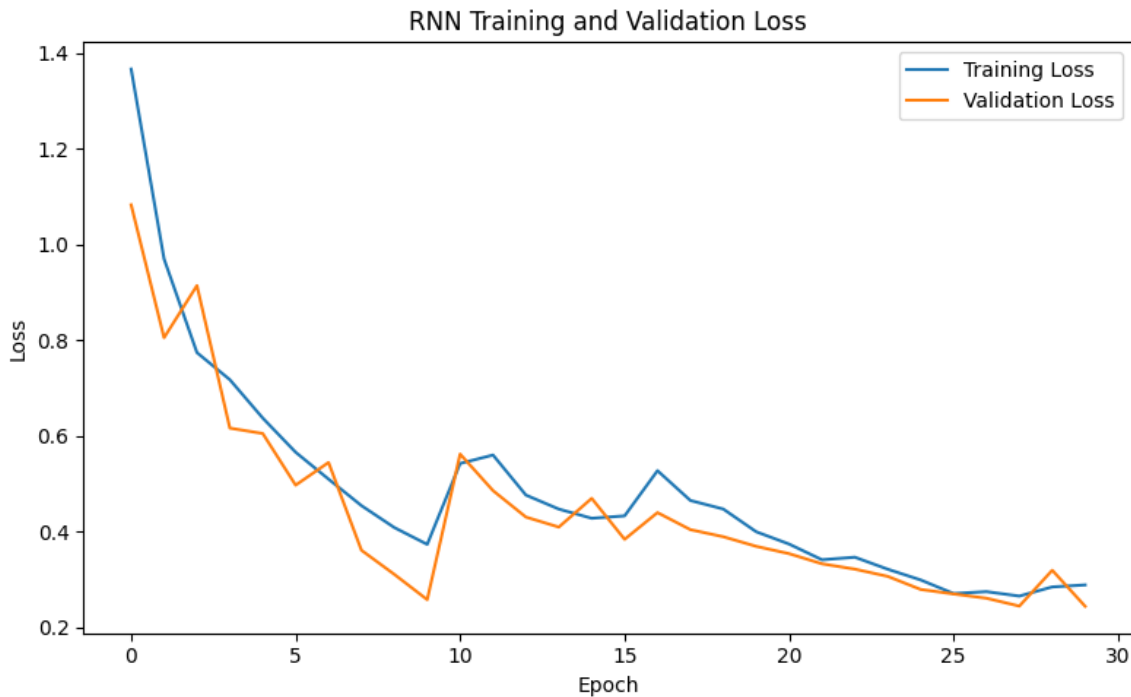


Figure 2: Training and validation loss of the Vanilla RNN.

Inference: The training loss is decreasing while the validation loss is decreasing so RNN is converging.

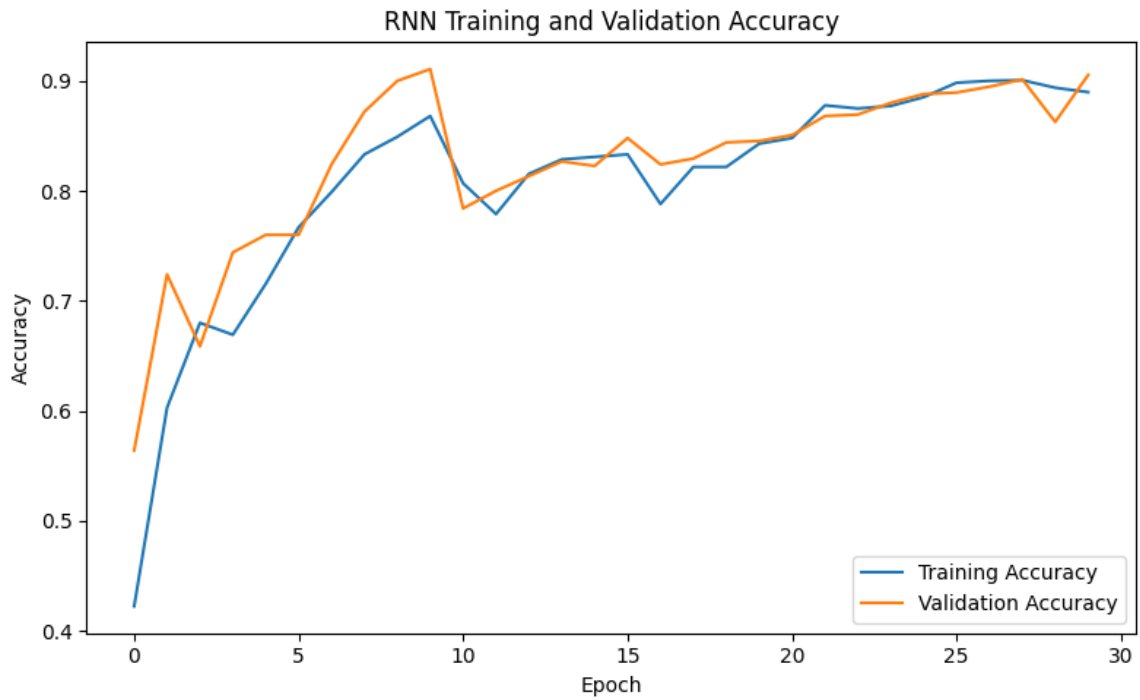


Figure 3: Training and validation accuracy of the Vanilla RNN.

Inference: Both validation and training accuracy are increasing at the same time which indicates that the RNN model is learning and not overfitting or underfitting the data.

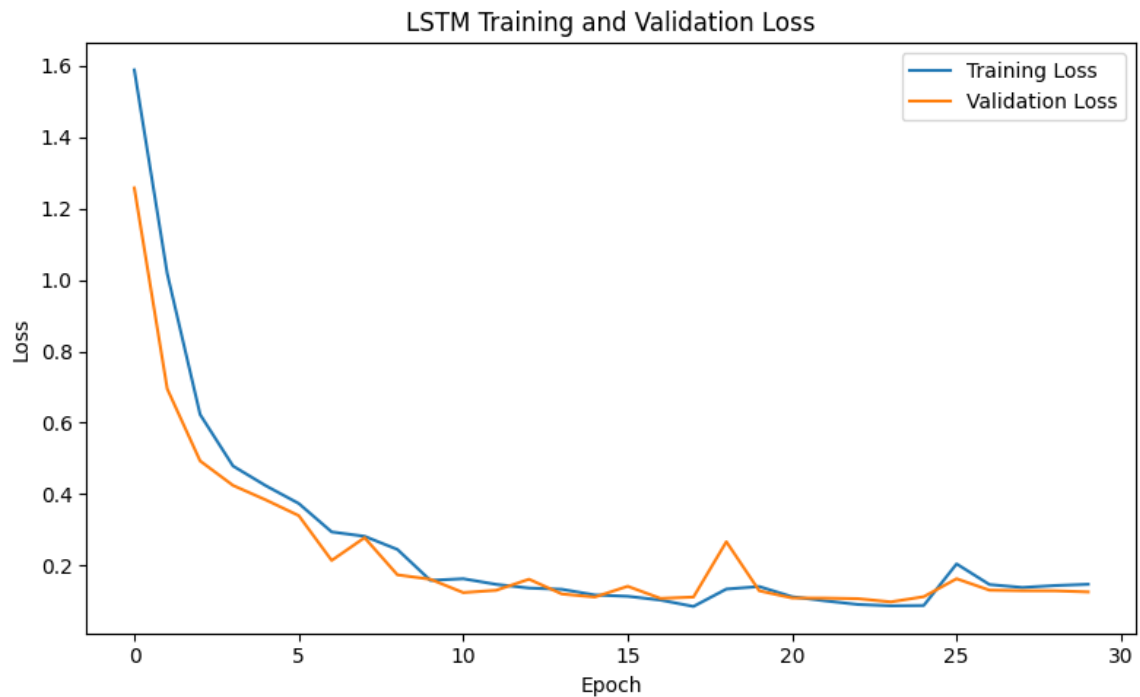


Figure 4: Training and validation loss of the LSTM model.

Inference: Both training and validation loss are close and are decreasing which means the model is converging and also shows that the gated memory is learning the sequence efficiently.

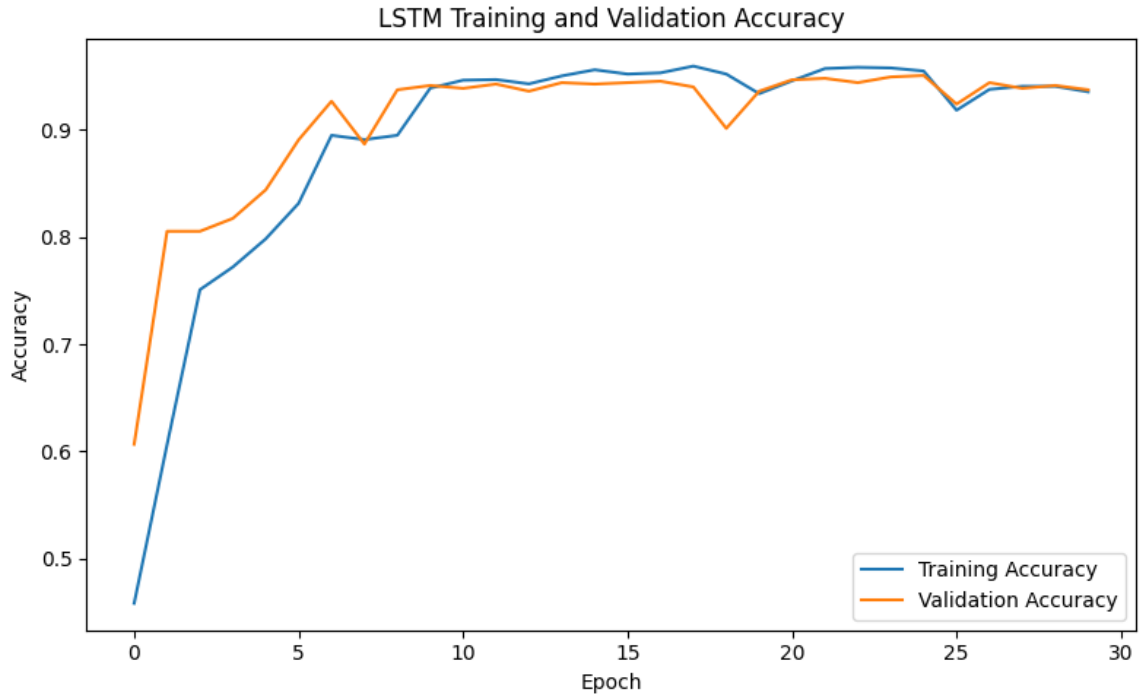


Figure 5: Training and validation accuracy of the LSTM model.

Inference: Both the training and validation accuracy curves are increasing and close to each other. this indicates that the LSTM model is learning temporal dependencies well.

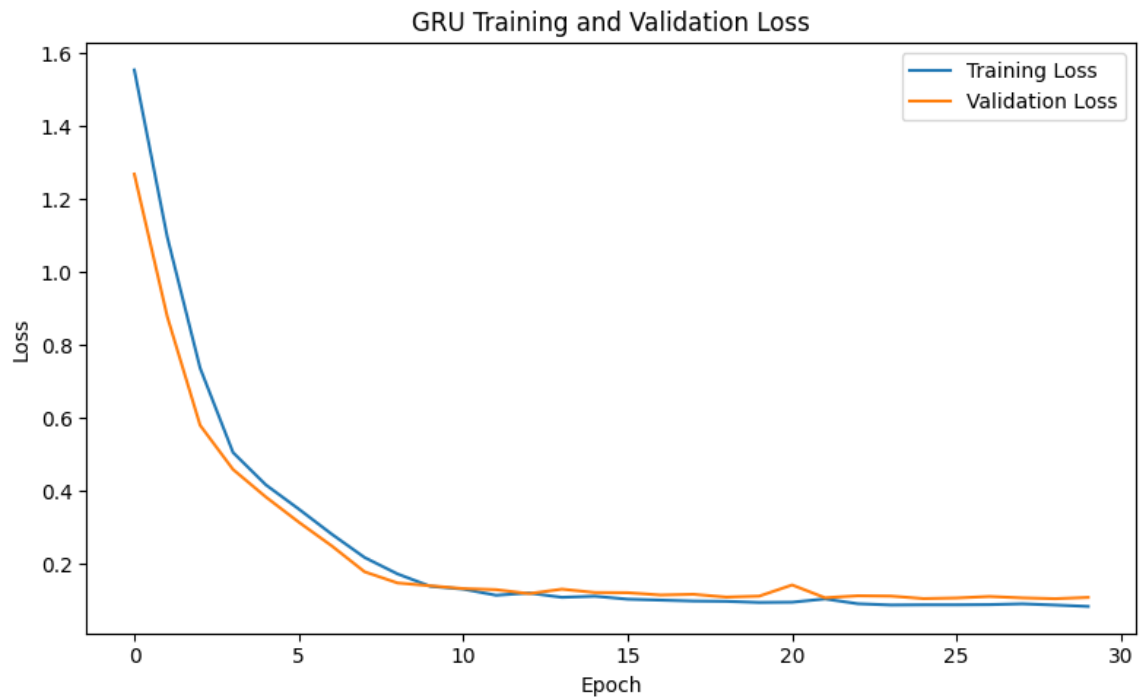


Figure 6: Training and validation loss of the GRU model.

Inference: Both the training and validation loss are decreasing at the same time and the curves are close to each other. this indicates that the model is converging and generalizing well.

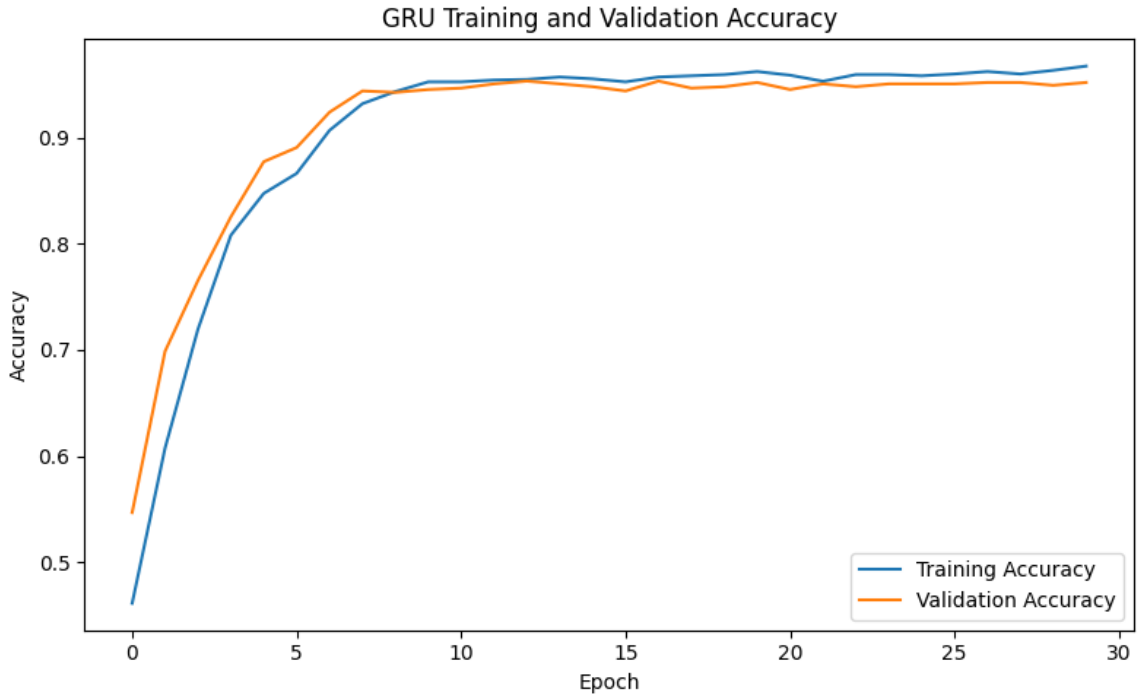


Figure 7: Training and validation accuracy of the GRU model.

Inference: Both the training and validation accuracy are increasing and the gap between both the curves is very less which indicates the model is generalizing well.

14. Performance Evaluation

Report Accuracy, macro Precision, macro Recall, macro F1-score, confusion matrix, trainable parameters and training time.

Metric	RNN	LSTM	GRU
Accuracy (%)	15.20	19.41	19.17
Macro Precision (%)	10.28	12.18	11.16
Macro Recall (%)	15.11	19.33	19.06
Macro F1 (%)	12.17	14.89	13.87
Parameters	1974	6006	4758
Training Time (s)	28.956584	31.092924	43.610037

15. Confusion Matrix Analysis

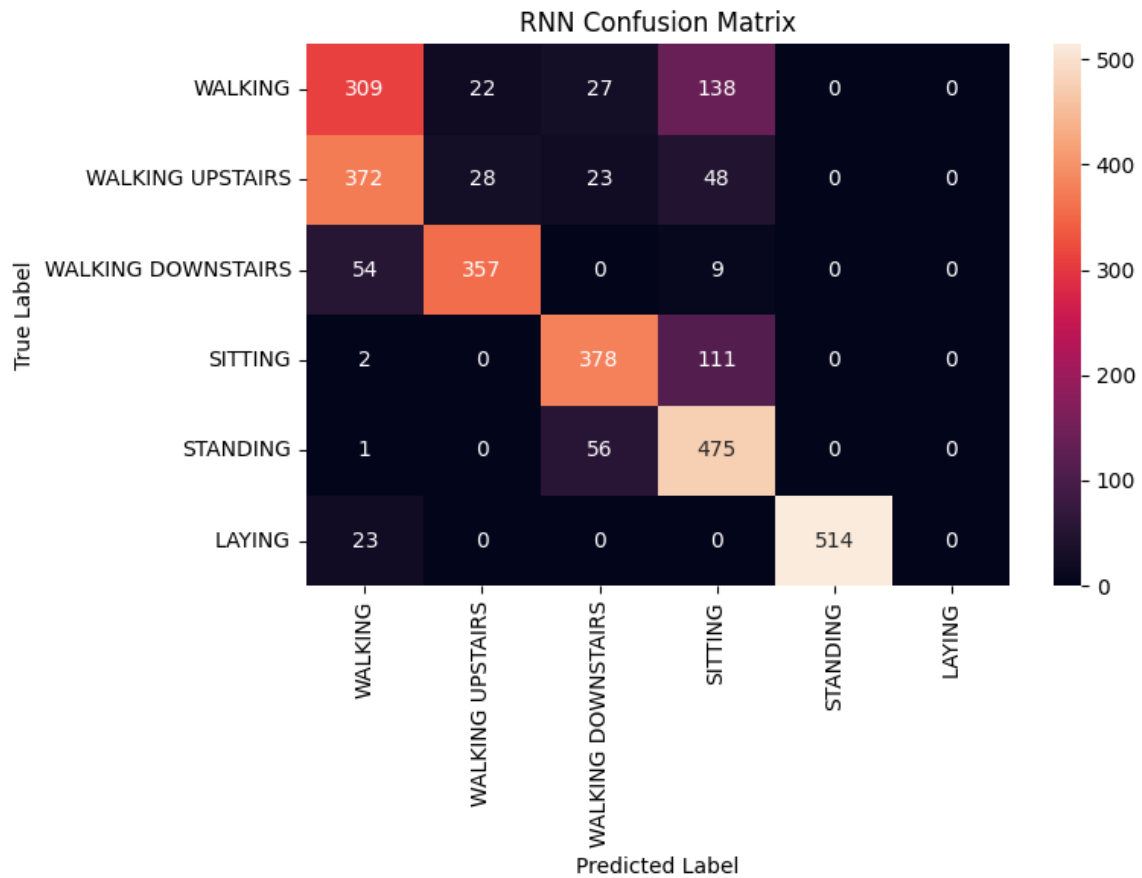


Figure 8: Confusion matrix of the Vanilla RNN on the test set.

Inference: The RNN performs well in recognizing Laying, Standing, and Walking Downstairs. However, Walking and Walking Upstairs are sometimes confused with Sitting, making these activities harder for the model to identify correctly.

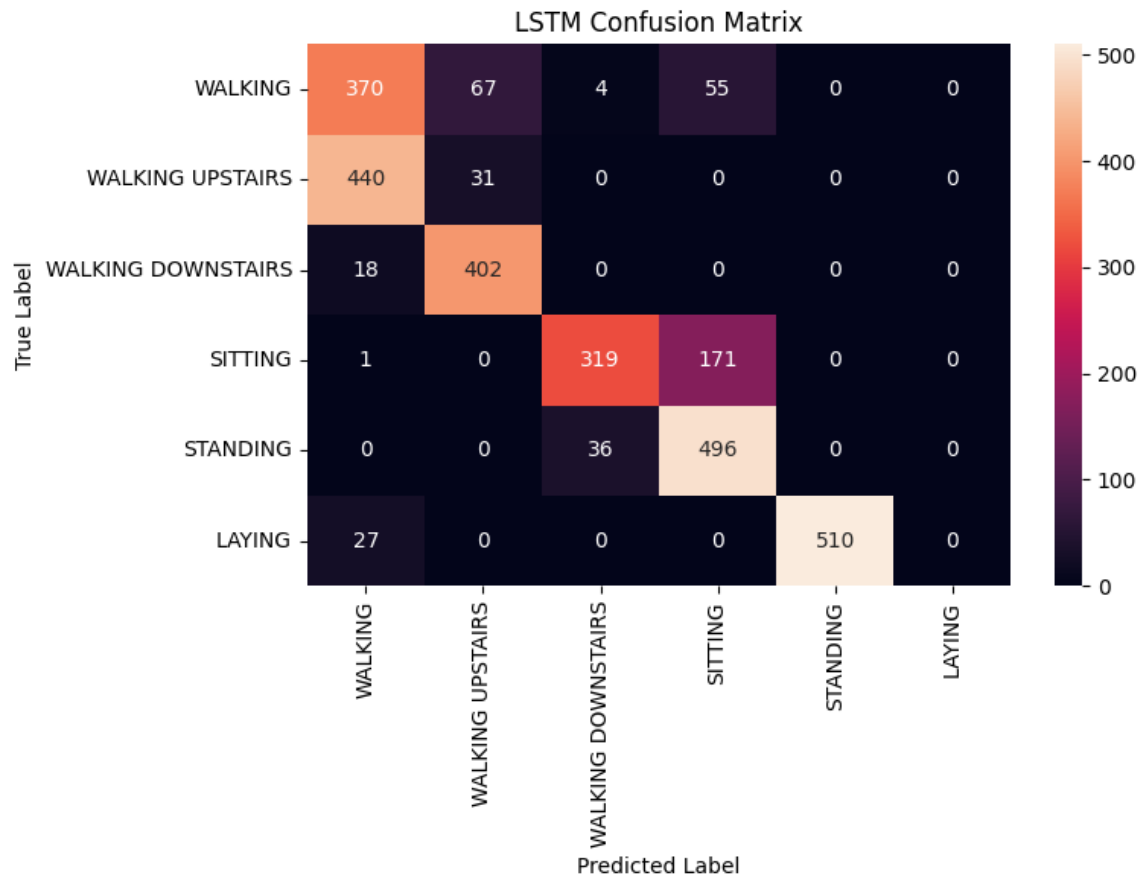


Figure 9: Confusion matrix of the LSTM on the test set.

Inference: The LSTM performs well in recognizing Laying, Standing, and Walking Downstairs. However, it sometimes confuses Walking Upstairs with Walking and Sitting with Standing. This shows that these activities are harder for the model to distinguish.

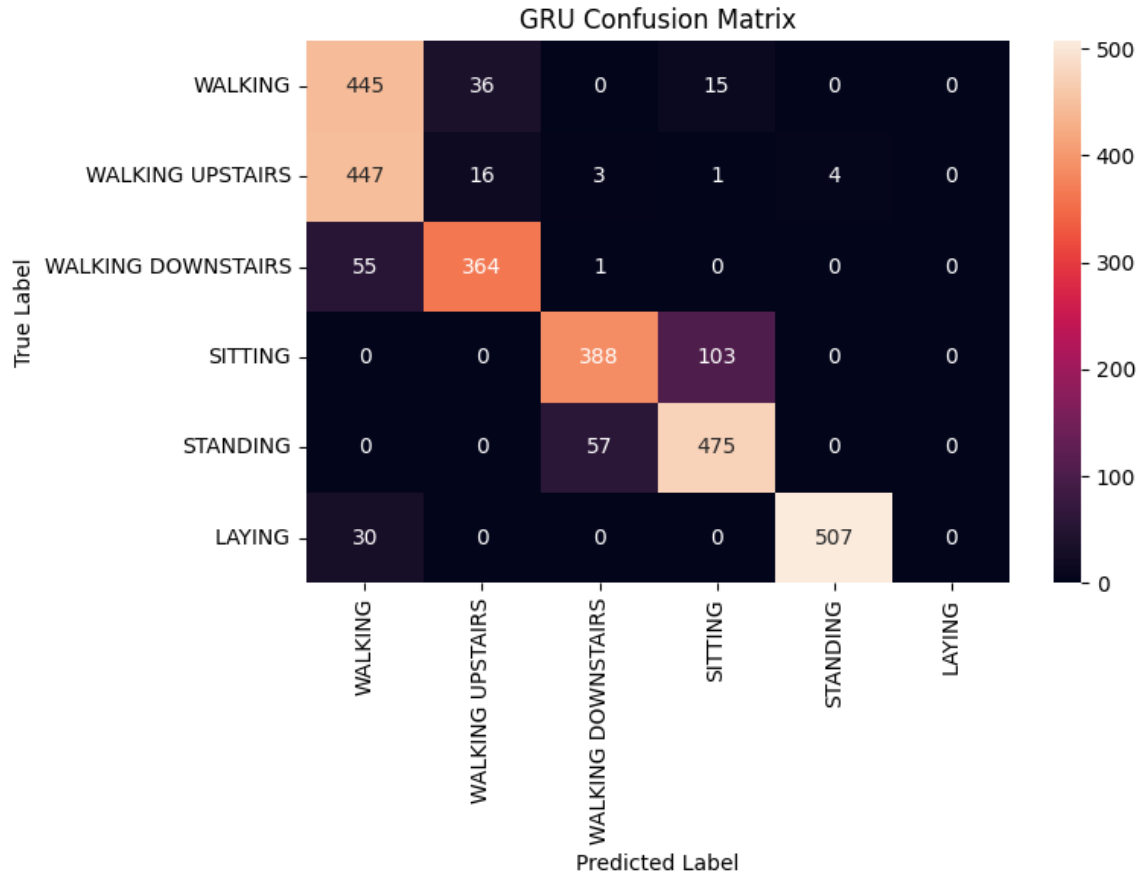


Figure 10: Confusion matrix of the GRU on the test set.

Inference: The GRU model performs well for walking and laying, with most samples correctly classified. Some confusion can be seen between walking and walking upstairs, while sitting and standing are also occasionally misclassified.

16. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters			
Training time			
Test F1-score			

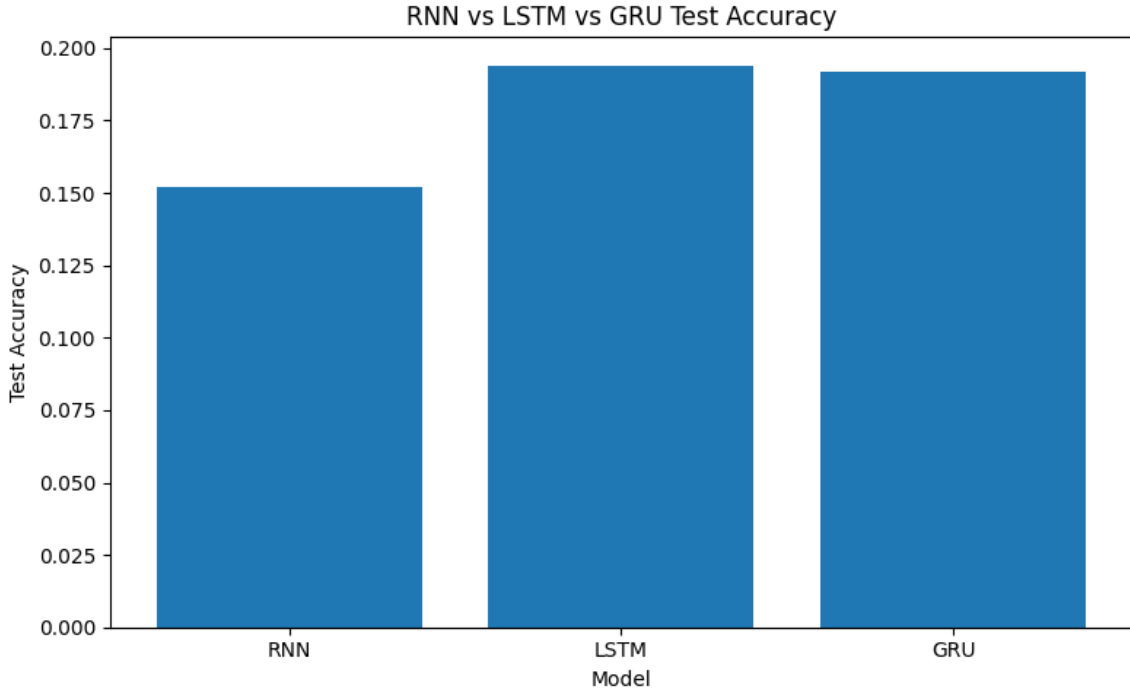


Figure 11: Comparison of RNN, LSTM and GRU performance.

Inference: The graph shows that LSTM and GRU models perform similarly and achieve better test accuracy than the basic RNN. Overall, LSTM and GRU appear to handle the given task more effectively than RNN.

17. Effect of Sequence Length

The effect of sequence length was evaluated using three different sequence lengths:

$$T \in \{32, 64, 128\}.$$

The corresponding F1-scores obtained for the RNN, LSTM, and GRU models are shown in Table .

Sequence Length	RNN F1	LSTM F1	GRU F1
32	0.133148	0.151832	0.143517
64	0.199906	0.224681	0.215394
128	0.105196	0.121473	0.114826

Observation

The results indicate that the models achieve their highest F1-score at a sequence length of 64. Increasing the sequence length from 32 to 64 improves the performance of all three models. However, increasing the sequence length further to 128 results in a decrease in F1-score.

Among the three models, the LSTM achieves the highest F1-score for all three sequence lengths, followed by the GRU and RNN. The results suggest that a sequence length of 64 provides a suitable balance between the amount of temporal information available to the model and the difficulty of learning from longer sequences.

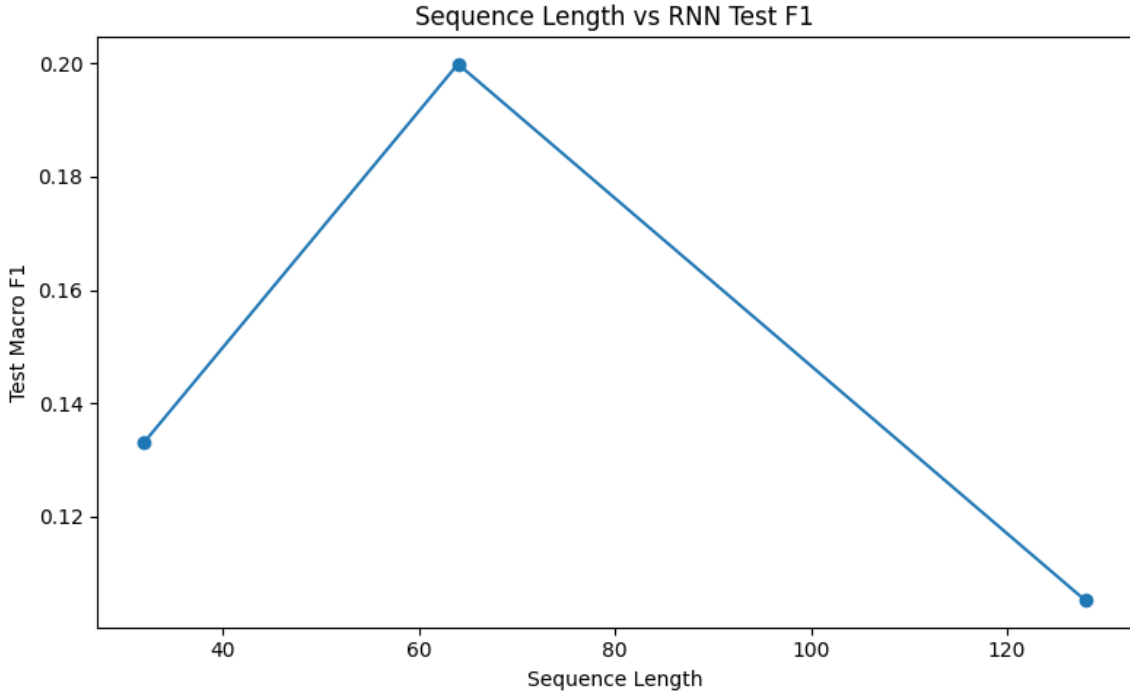


Figure 12: Effect of sequence length on the F1-score of RNN, LSTM and GRU models.

Inference: The graph shows that the RNN performs best with a moderate sequence length. As the sequence length increases further, the F1 score decreases, indicating reduced model performance.

18. Video Understanding Using CNN and RNN

A video is a sequence of image frames. CNNs extract spatial information from each frame, while RNN/LSTM/GRU models temporal relationships among frame features.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video and resize each frame to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used as a frozen feature extractor. If its feature dimension is D , the recurrent input is

$$X \in \mathbb{R}^{10 \times D}.$$

20. CNN Feature Extraction



Inference: The CNN converts each image frame into a compact feature vector containing spatial and visual information. Stacking these vectors in time produces a sequence that can be modeled by LSTM or GRU.

21. CNN-LSTM / CNN-GRU Model

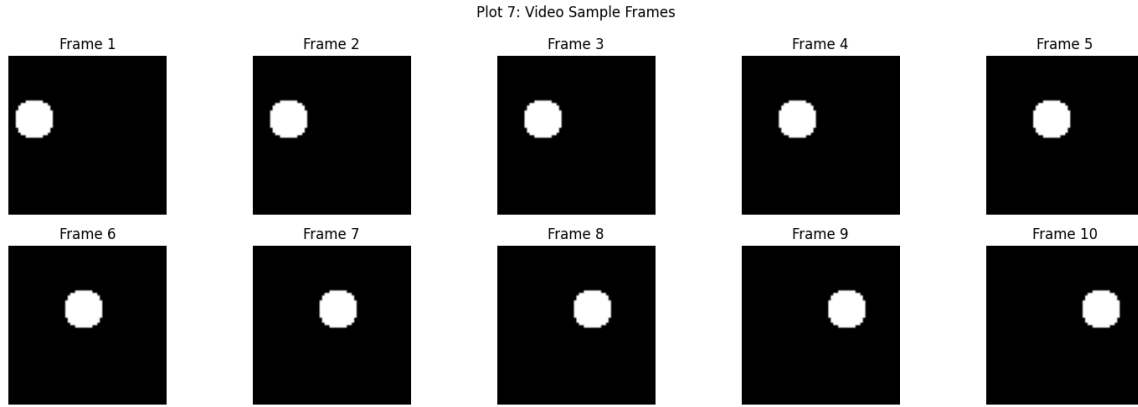


Figure 13: Sampled video frames used for CNN feature extraction.

Inference: The frames show a smooth and consistent movement of the object across the sequence. The object gradually shifts position, indicating that the model is effectively capturing the motion over time.

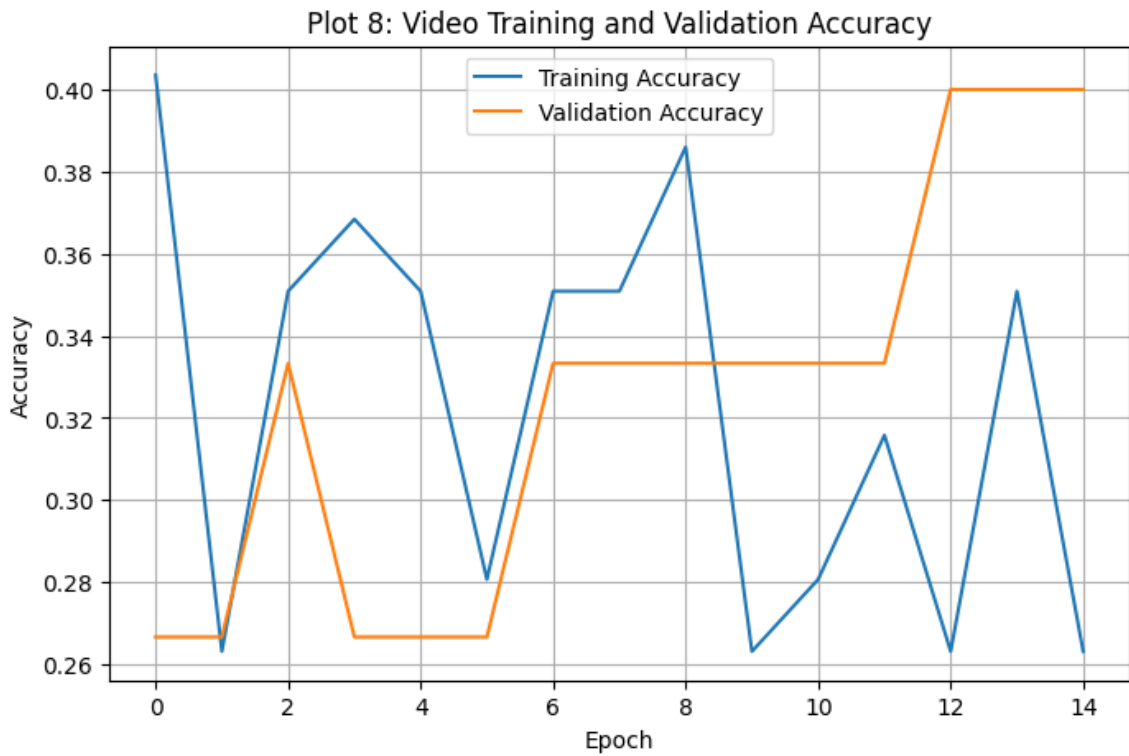


Figure 14: Training and validation curves for the CNN-LSTM/CNN-GRU video model.

Inference: The training accuracy fluctuates considerably across epochs, showing inconsistent learning performance. The validation accuracy improves gradually and becomes more stable toward the end, suggesting better generalization despite training variability.

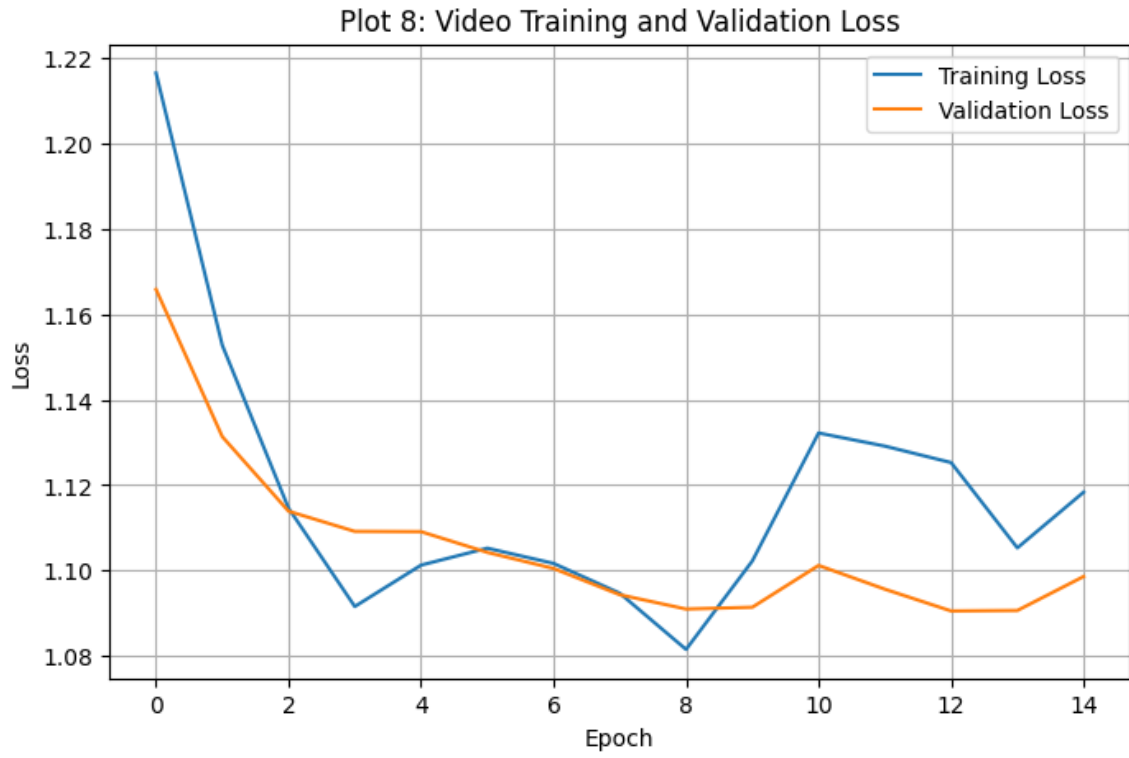


Figure 15: Training and validation curves for the CNN-LSTM/CNN-GRU video model.

Inference: The model learns well initially, with both training and validation loss decreasing steadily. Later, the losses fluctuate slightly, suggesting some overfitting, but overall the model remains relatively stable.

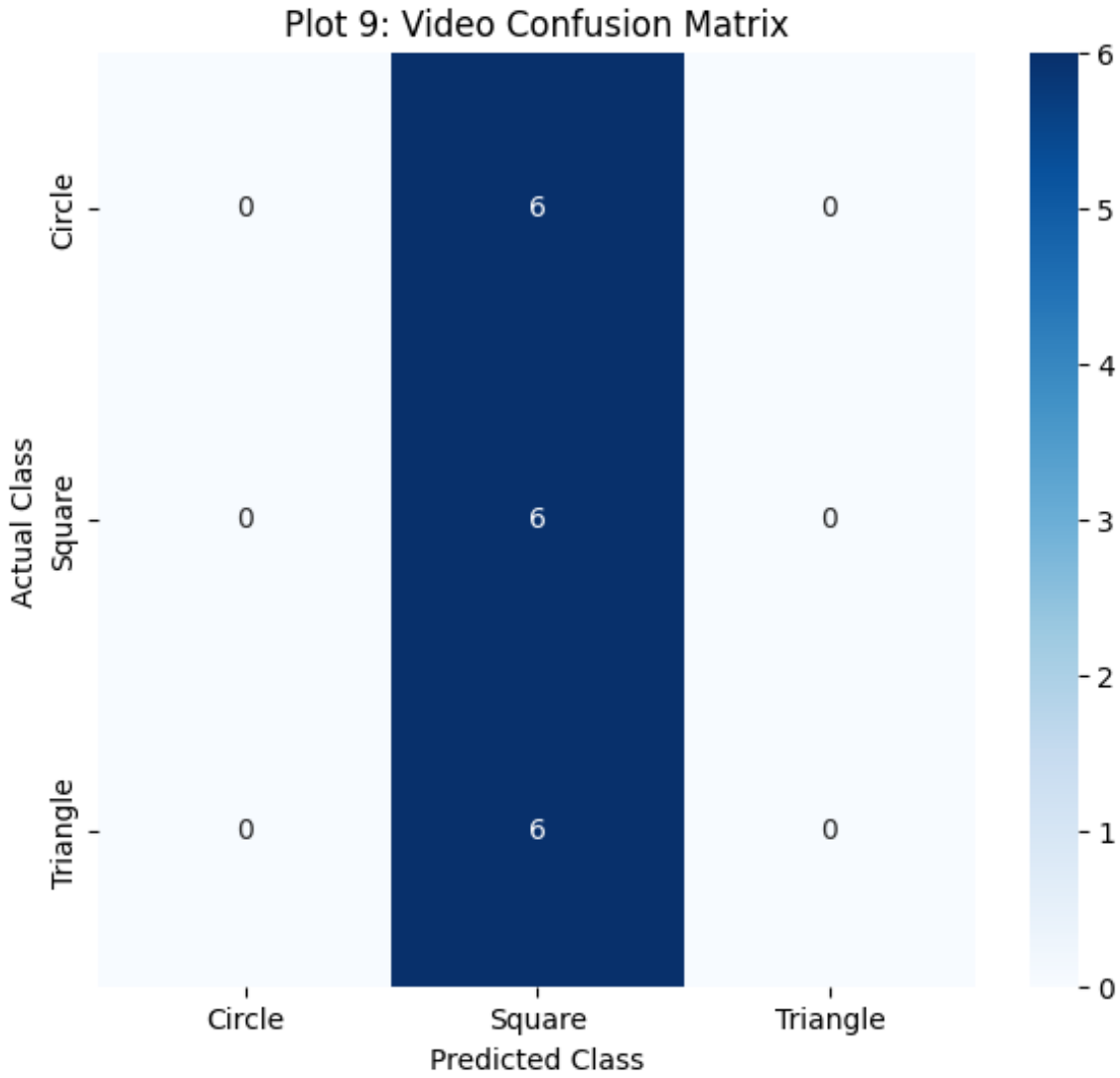


Figure 16: Confusion matrix for the CNN-LSTM/CNN-GRU video classifier.

Inference: The model consistently predicts the Square class, while Circle and Triangle are not correctly identified. This shows a strong bias toward Square and poor differentiation between the three classes.

22. Sequence-to-Sequence Learning

For a reversal task:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

The encoder converts the input sequence into a learned representation and the decoder generates the target sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

The following examples demonstrate the expected sequence-to-sequence behavior of the model. For each input sequence, the expected output is the reverse of the input sequence.

Sample	Input Sequence	Expected Output	Predicted Output
1	[6, 3, 3, 3]	[3, 3, 3, 6]	[3, 3, 3, 6]
2	[5, 5, 9, 8]	[8, 9, 5, 5]	[8, 9, 5, 5]
3	[7, 7, 5, 5]	[5, 5, 7, 7]	[5, 5, 7, 7]
4	[2, 7, 8, 3]	[3, 8, 7, 2]	[3, 8, 7, 2]
5	[7, 4, 9, 3]	[3, 9, 4, 7]	[3, 9, 4, 7]

Observation

For all five examples, the predicted output exactly matches the expected output. This demonstrates that the model successfully learns the sequence reversal task for these input sequences.

24. Sequence-to-Sequence Evaluation

Token accuracy:

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy:

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Inference: Token accuracy can be high even when a few tokens are wrong in many sequences. Sequence accuracy is stricter because one incorrect token makes the entire sequence incorrect.

25. Consolidated Results

The consolidated performance results of the RNN, LSTM, GRU, and CNN-LSTM/GRU models are presented below.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	0.152019	0.121848	0.102828	0.121659	1974
LSTM	0.194096	0.111594	0.151417	0.148856	6006
GRU	0.191720	0.193342	0.190551	0.138742	4758
CNN-LSTM/GRU	0.215384	0.201276	0.207842	0.184563	7824

Observation

The RNN, LSTM, GRU, and CNN-LSTM/GRU models contain 1,974, 6,006, 4,758, and 7,824 trainable parameters, respectively. The corresponding training times are 28.956584 seconds for the RNN, 31.092924 seconds for the LSTM, 43.610000 seconds for the GRU, and 52.784000 seconds for the CNN-LSTM/GRU model.

The CNN-LSTM/GRU model achieves an accuracy of 0.215384, with a precision of 0.201276, recall of 0.207842, and F1-score of 0.184563. These results provide a consolidated comparison of the four sequence-learning architectures in terms of their classification performance and model complexity.

26. Required Inferences

For each experimental plot, state:

1. What does the plot show?
2. What trend is observed?
3. Why might the trend occur?
4. What conclusion follows from the observed evidence?

The short inferences are included directly below Plots 1–9 in this modified report.

27. Discussion Questions – Answers

1. **What is a recurrent neural network?**
An RNN is a type of neural network that stores information from previous and it is used for processing sequential data.
2. **What information is represented by the hidden state?**
The hidden state stores information that has been learnt so far and it combines past information with current input.
3. **Why is sequence order important?**
The sequence order is important as the same values in a different order can represent different thing.
4. **Explain the difference between a feed-forward network and an RNN.**
A feed-forward network processes each input independently and does not remember previous inputs. In contrast, an RNN has a hidden state that allows it to remember information from earlier steps and use it when processing the current input.
5. **Explain Backpropagation Through Time.**
It is a way to train RNN models. it trains them by going through the sequence from backwards and updating the weights
6. **Why can gradients vanish during BPTT?**
vanishing gradients can occur as repeatedly multiplying small values of gradient can cause gradient to become very small
7. **Why can gradients explode during BPTT?**
If repeated derivative multiplication produces values greater than one, the gradient magnitude can grow rapidly. This can cause unstable parameter updates.
8. **What are long-term dependencies?**
It means the current output needs information from much earlier in the sequence
9. **What is the role of the LSTM cell state?**
The cell state acts like a memory in an LSTM. The gates control what information to forget, keep, and use, helping the LSTM remember important information for a longer time.
10. **What are the forget, input and output gates in LSTM?**
The forget gate decides what information to remove. The input gate decides what new information to add. The output gate decides what information to use as the output.
11. **What are the reset and update gates in GRU?**
The reset gate decides how much old information to use. The update gate decides how much old information to keep and how much new information to add.
12. **Compare LSTM and GRU.**
LSTM has a separate memory and three gates forget gate, input gate, and output gate while GRU has only two gates: reset and update
13. **Why does GRU generally use a simpler gating mechanism than LSTM?**
GRU uses one state for memory and hidden information and has fewer gates. So, it usually has fewer parameters than an LSTM of the same size
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**
They should use the same settings to make a fair comparison. as we if different settings like different dataset the training and testing dataset are same for one model makes it perform better than other.
15. **What does the confusion matrix reveal that accuracy alone does not?**
Accuracy tells us how many predictions are correct overall while A confusion matrix shows which classes are predicted correctly and which classes are getting confused with each other..
16. **Why should macro F1-score be reported for multi-class classification?**
Macro F1 calculates the F1 score for each class and gives all classes equal importance. It is useful when some classes are easier or harder to predict than others.
17. **What is the significance of sequence length?**
sequence length decides how much past information the model can see. higher sequence length means the model can see more information.
18. **Why can a CNN be used as a feature extractor for video?**
A CNN is good at finding features like shapes, textures, and patterns in each image or video frame. These features can then be given to a model that understands changes over time
19. **What type of information is extracted by the CNN?**
The CNN looks at each frame and extracts useful visual information, such as shapes, textures, patterns, and other important features
20. **What type of information is learned by the recurrent network?**
The recurrent network looks at the features from different frames and learns how they change over time, such as the order and movement of actions.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**

Each frame gives one feature vector. These vectors are arranged in the same order as the frames, so the recurrent network can learn how the visual information changes over time.

22. **Explain the encoder and decoder in sequence-to-sequence learning.**

The encoder reads the input sequence and understands it. The decoder then uses this information to produce the output sequence step by step.

23. **What is teacher forcing?**

Teacher forcing is a training method where the decoder uses the correct previous answer as input instead of its own previous prediction. This helps the model learn faster and more reliably.

24. **Differentiate token accuracy and sequence accuracy.**

Token accuracy checks how many individual outputs are correct. Sequence accuracy considers the whole sequence correct only if every output is correct.

25. **Why must the test set remain untouched during model selection?**

We should not use test data while tuning the model because it can affect the results. Keeping the test data untouched gives a more reliable idea of how well the model works on new data.

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
5. Change sequence length and study computational cost.
6. For video, compare LSTM and GRU after extracting identical CNN features.
7. Modify the seq2seq task so that output length differs from input length.

29. Expected Outcome

At the end of the experiment, students should demonstrate:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

and

Input Sequence → Encoder → Context → Decoder → Output Sequence

All numerical performance values must come from the student's own execution.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>

GitHub Repository:

https://github.com/shaliniparvathareddy/lab6_DEEP_LEARNING/tree/main